

Ciencia de Datos

Fundamentos de aprendizaje automático

¿Qué es el Machine Learning?

El **Machine Learning (ML)** es una rama de la inteligencia artificial que permite a los sistemas **aprender patrones a partir de datos** y mejorar su desempeño sin ser programados explícitamente.

- No se escriben reglas manualmente — el modelo las **descubre**
- Más datos + mejor algoritmo = **mejores predicciones**
- Es la base del análisis predictivo moderno

ML vs. Programación tradicional

Programación tradicional	Machine Learning
Humano define las reglas	El modelo aprende las reglas
Entrada + Reglas → Salida	Entrada + Salida → Reglas
Frágil ante datos nuevos	Se generaliza a datos no vistos
Apropiada para lógica fija	Apropiada para patrones complejos

En ML le damos **ejemplos**, no instrucciones.

Tipos de aprendizaje

Tres grandes paradigmas

 **Aprendizaje supervisado** — se entrena con datos etiquetados (entrada + respuesta correcta)

 **Aprendizaje no supervisado** — el modelo descubre estructura en datos sin etiquetas

 **Aprendizaje por refuerzo** — un agente aprende por prueba y error recibiendo recompensas

Supervisado vs. No supervisado

Característica	Supervisado	No supervisado
Etiquetas	✓ Necesita etiquetas	✗ Sin etiquetas
Objetivo	Predecir una salida	Encontrar grupos/patronos
Ejemplos de tareas	Clasificación, regresión	Clustering, reducción de dimensiones
Algoritmos típicos	SVM, Naive Bayes, Redes Neuronales	K-Means, PCA

Ciclo de vida de un modelo

Las etapas del proyecto ML

1. **Definir el problema** — ¿qué queremos predecir o descubrir?
2. **Recolectar y explorar datos** — EDA, calidad de datos
3. **Preprocesar** — limpieza, codificación, escalado
4. **Seleccionar y entrenar el modelo** — split train/test, fit
5. **Evaluar** — métricas de desempeño
6. **Iterar y mejorar** — ajuste de hiperparámetros
7. **Desplegar** — integración al sistema productivo

División de datos: train / test

 **Conjunto de entrenamiento (train)** — el modelo aprende con estos datos (típicamente 70–80 %)

 **Conjunto de prueba (test)** — se evalúa el desempeño con datos nunca vistos (20–30 %)

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

scikit-learn: la librería central

¿Por qué scikit-learn?

- Biblioteca de ML más usada en Python
- API consistente: `fit` → `predict` → `score`
- Incluye algoritmos de clasificación, regresión, clustering y más
- Integración nativa con `pandas` y `numpy`

```
import sklearn  
print(sklearn.__version__)
```

La API unificada de scikit-learn

```
from sklearn.linear_model import LinearRegression

# 1. Instanciar el modelo
modelo = LinearRegression()

# 2. Entrenar (aprender de los datos)
modelo.fit(X_train, y_train)

# 3. Predecir
y_pred = modelo.predict(X_test)

# 4. Evaluar
score = modelo.score(X_test, y_test)
print(f"R² en test: {score:.4f}")
```

Este patrón **fit** → **predict** → **score** se repite en casi todos los modelos de scikit-learn.

Preprocesamiento con scikit-learn

```
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import LabelEncoder

# Escalado de características numéricas
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)    # ojo: solo transform, no fit

# Codificación de variables categóricas
le = LabelEncoder()
y_encoded = le.fit_transform(y_categorica)
```

 **Regla importante:** el scaler se ajusta (`fit`) solo con datos de entrenamiento; al test solo se aplica (`transform`).

Conceptos clave del aprendizaje

Underfitting y Overfitting

Situación	Descripción	Síntoma
Underfitting	El modelo es demasiado simple	Error alto en train y en test
Ajuste correcto	El modelo generaliza bien	Error bajo en train y test similares
Overfitting	El modelo memoriza el train	Error bajo en train, alto en test

El objetivo es encontrar el **balance óptimo** entre sesgo y varianza.

Hiperparámetros vs. Parámetros



Parámetros — los aprende el modelo durante el entrenamiento (ej. coeficientes de regresión, pesos de una red neuronal)



Hiperparámetros — los define el programador antes de entrenar (ej. número de vecinos en KNN, profundidad de un árbol, tasa de aprendizaje)

```
from sklearn.neighbors import KNeighborsClassifier

# n_neighbors es un hiperparámetro
knn = KNeighborsClassifier(n_neighbors=5)
```


Resumen de la semana

Concepto	Definición breve
Machine Learning	Sistemas que aprenden patrones desde datos
Aprendizaje supervisado	Datos etiquetados → predecir salida
Aprendizaje no supervisado	Datos sin etiquetas → encontrar estructura
Train / Test split	Separar datos para entrenar y evaluar
scikit-learn	Librería Python con API fit/predict/score
Overfitting	Memorizar train, fallar en test
Hiperparámetros	Parámetros de configuración definidos por el programador

Para esta semana

- [] Comprender la diferencia entre ML supervisado y no supervisado
- [] Identificar las etapas del ciclo de vida de un modelo ML
- [] Aplicar `train_test_split` para dividir un dataset
- [] Entrenar un primer modelo con la API `fit → predict → score` de `scikit-learn`
- [] Identificar casos de underfitting y overfitting en gráficas de error

Próxima sesión

Clasificación: Redes Neuronales básicas · Naive Bayes · Pronóstico con SVM ·
entrenamiento con datos etiquetados